# Use the Conveyor Belt

## With the Control Box



The rotary potentiometer allows you to control the speed and the direction of your conveyor.
When the potentiometer is at its origin, centered, the Conveyor Belt is stopped.
The more you turn the potentiometer clockwise, the fastest the Conveyor Belt will move forward.
Conversely, the more you turn the potentiometer counterclockwise, the fastest the Conveyor Belt will move backward.

## With NiryoStudio

You can have all the details about the application in NiryoStudio's documentation.

In NiryoStudio, open the Equipment tab in the left menu. The interface will let you control up to two Conveyors directly. You can control their speed and direction.

> Click on **Scan** to detect and add the connected Conveyor Belt to NiryoStudio.



### IR sensor

Connect your sensor on IR-Sensor connector on the back panel of the robot. In NiryoStudio, in the “Digital I/O panel”, you can read the IR-Sensor state in **DI5** pin:

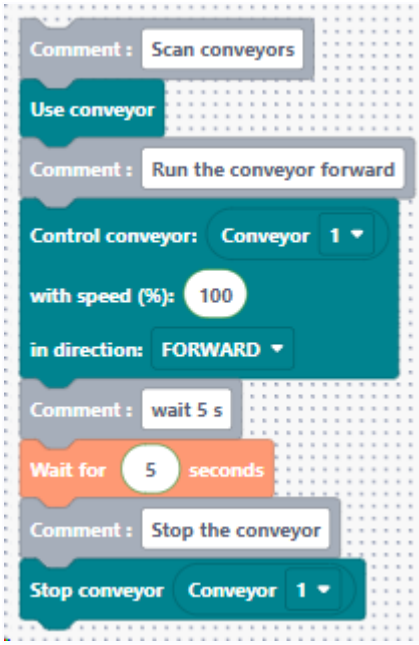
> When the distance measured is **smaller** than the adjusted distance, the “INPUT” is “LOW”, it means an obstacle is detected.
> When the distance measured is **higher** than the adjusted distance, the “INPUT” is “HIGH”, it means that there is no obstacle.



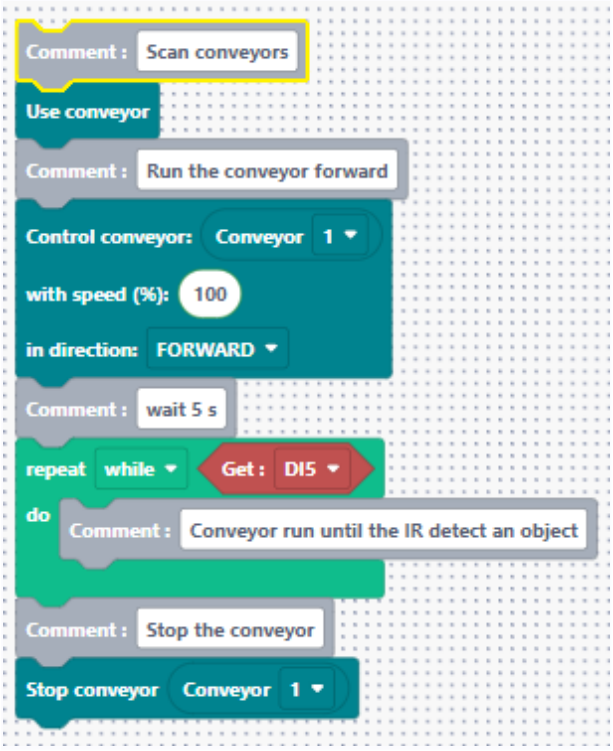

## With Blockly

Special blocks have been designed to be used with the Conveyor. These blocks keep Ned2’s Ecosystem easy to use with NiryoStudio.

The following code shows how to scan the conveyor and run it Forward.




The following code shows how to scan the conveyor and run it Forward until the IR detect an object. Please refer to the Infrared Sensor documentation for more information about the states.




## With PyNiryo

Here are provided two scripts for the Conveyor Belt and the IR sensor. You can find more details in PyNiryo's Documentation.

## PyNiryo - The Conveyor Belt

Here is a code example showing how to scan the connected Conveyor Belt and run it forward.

**python**

```python
#!/usr/bin/env python

from pyniryo import *

# Connecting to robot
robot = NiryoRobot(<robot_ip_address>)

# Activating connexion with Conveyor Belt
conveyor_id = robot.set_conveyor()

# Running the Conveyor Belt (v2) at 50% of its maximum speed, in forward direction
robot.run_conveyor(conveyor_id, speed=50, direction=ConveyorDirection.FORWARD)

# Waiting 3 seconds
robot.wait(3)

# Stopping conveyor's motor
robot.stop_conveyor(conveyor_id)

# Deactivating connexion with the Conveyor Belt
robot.unset_conveyor(conveyor_id)
```

## PyNiryo - IR sensor

You can get the IR sensor state and control the Conveyor Belt accordingly as follows. Please note that, in this example, the sensor is connected to the DI5.

```
#!/usr/bin/env python

from pyniryo import *

# Connecting to the robot
robot = NiryoRobot(<robot_ip_address>)

# Activating connexion with the Conveyor Belt
conveyor_id = robot.set_conveyor()

# -- Setting variables
sensor_pin_id = PinID.DI5
# Run conveyor and wait until the IR sensor detects an object
robot.run_conveyor(conveyor_id)
while robot.digital_read(sensor_pin_id) == PinState.HIGH:
    robot.wait(0.1)

# Stopping conveyor's motor
robot.stop_conveyor(conveyor_id)
```

```
# Deactivating connexion with the Conveyor Belt
robot.unset_conveyor(conveyor_id)
```

## With Python ROS Wrapper

More details about how to use the Python ROS Wrapper with Ned2 and the Convevor Belt can be found the Python ROS wrapper documentation.

> If you are **using a physical Ned2** and you don't want to write the following code directly on the robot, you will need to setup a **multi-machines** environment. On the other hand, you can directly use PyNiryo following the PyNiryo section.

## Python ROS Wrapper - The Conveyor Belt

Here is a code example on how to control the Conveyor through the Python ROS Wrapper:

**python**

```python
#!/usr/bin/env python

# Imports
from niryo_robot_python_ros_wrapper import *
import rospy

niryo_robot = NiryoRosWrapper()
rospy.init_node('niryo_blockly_interpreted_code')

# Activating connexion with conveyor and storing ID
conveyor_id = niryo_robot.set_conveyor()

# Running conveyor at 50% of its maximum speed, in Forward direction
niryo_robot.control_conveyor(ConveyorID.ID_1, True, 100, ConveyorDirection.FORWARD

# Wait 3 seconds
niryo_robot.wait(3)

# Stopping conveyor's motor
niryo_robot.control_conveyor(ConveyorID.ID_1, True, 0, ConveyorDirection.FORWARD)

# Deactivating connexion with conveyor
niryo_robot.unset_conveyor(conveyor_id)
```

## Python ROS Wrapper - IR sensor

You can get the IR sensor state and control the Conveyor Belt accordingly as follows. Here, the Conveyor Belt is connected to the DI5.

**python**

```python
#!/usr/bin/env python

# Imports
from niryo_robot_python_ros_wrapper import *
import rospy

sensor_pin_id = PinID.DI5

niryo_robot = NiryoRosWrapper()
rospy.init_node('niryo_blockly_interpreted_code')

# Activating connexion with conveyor
niryo_robot.set_conveyor()
conveyor_id = ConveyorID.ID_1

# Run conveyor and wait until the IR sensor detects an object
niryo_robot.control_conveyor(conveyor_id,True,speed=50,direction=ConveyorDirection

while niryo_robot.digital_read(sensor_pin_id) == PinState.HIGH:
    niryo_robot.wait(0.1)

# Stopping conveyor's motor
niryo_robot.control_conveyor(conveyor_id, True, 0, ConveyorDirection.FORWARD)

# Deactivating connexion with conveyor
niryo_robot.unset_conveyor(conveyor_id)
```

---

## With Modbus

**python**

```python
from pymodbus.client import ModbusTcpClient
import time

if __name__ == '__main__':
    print("--- START")
    client = ModbusTcpClient('10.10.10.10', port=5020)
    client.connect()
    print("Connected to modbus server")

    # attach the conveyor 1
    print("Ping and set the conveyor")
    result = client.read_coils(53)
    if not result.bits[0]:
```

```
        print('scanning convyeor')
        client.write_coil(53, True)

    time.sleep(2)
    # set the conveyor 1 to backward
    print("set the direction to backward")
    client.write_coil(93, 0)
    time.sleep(2)

    # set the conveyor 1 speed to 50
    print("set the speed to 50% ans start")
    client.write_register(87, 50)
    time.sleep(2)

    print("stop the conveyor")
    client.write_coil(73, 0)
    time.sleep(2)

    print("set the direction to forward")
    client.write_coil(93, 1)
    time.sleep(2)

    print("set the speed to 100% ans start")
    client.write_register(87, 100)
    time.sleep(2)

    print("stop the conveyor plugged")
    client.write_coil(73, 0)
    time.sleep(1)

    print("Remove the conveyor")
    client.write_coil(53, 0)
    time.sleep(1)

    client.close()
    print("Close connection to modbus server")
    print("--- END")
```

Made with Docsie.io